Terrain-Aware Autonomous Landing Gear Control for UAVs: Multi-Sensor Fusion, Active Attitude Stabilisation and Edge-AI Terrain Classification

Simulation and Hardware Bench Prototypes

Manitsas Andreas Vrana Maria

Aristotle University of Thessaloniki
Course: UAV01 – Sensor Systems for UAVs

October 2026

Abstract

Landing is the most hazardous phase of unmanned aerial vehicle (UAV) operation, and an undercarriage that is deployed over unsuitable terrain (a steep slope, a stepped surface or an obstructed patch) can cause rollover or structural damage. This report presents the hardware concept and the methods of a landing gear assistance system that judges, during terminal descent, whether the terrain below is suitable for landing and gates gear deployment accordingly. Two complementary prototypes are described. The first is a ROS 2 and Gazebo simulation in which three time-of-flight (ToF) rangefinders, one ultrasonic rangefinder and an inertial unit feed a seven-element feature vector; a decision tree (maximum depth four, trained on 1 500 labelled synthetic samples) turns the vector into a verdict, and an abort flag overrides the gear request through a rate-limited gear controller. Terrain slope is obtained from the range differences across the ToF triangle, and attitude from a complementary filter driving a PID pitch stabiliser. The second prototype is an Arduino Uno bench build with a real inertial unit and a single ToF sensor that exercises attitude estimation, a virtual stabiliser command, rule-based landing-readiness checks and a latched gear sequence with the servos disconnected. In simulation, all three static terrain worlds produce the expected verdict, abort flag and gear status, and the classifier scores 375 of 375 on a held-out split; the report explains why this score is optimistic, since the classes form tight clusters and the thresholds lie midway between them. A host benchmark gives a mean inference time of 21.98 ns, which is a host measurement and not a bound for an embedded target. The report states the limits of the evidence throughout.

Contents

1 Introduction **2**
1.1 Problem statement . . . 2
1.2 Objectives and contributions . . . 3
1.3 Organisation of the report . . . 3

2 System Architecture **3**

3 Hardware **4**
3.1 Embedded computing targets . . . 4
3.2 Sensors . . . 4
3.3 Actuators . . . 6
3.4 Simulated airframe . . . 6

4 Algorithms and Methods **6**
4.1 Attitude estimation by complementary filtering . . . 7
4.2 Active pitch stabilisation . . . 8
4.3 Terrain slope from the ToF triangle . . . 9
4.4 Optical-acoustic comparison . . . 10
4.5 Feature extraction . . . 10
4.6 Edge-AI terrain classifier . . . 11
4.7 Gear control and safety interlock . . . 16
4.8 Perception-driven approach supervisor . . . 17

5 Software Implementation **18**
5.1 Platform . . . 18
5.2 Nodes . . . 18
5.3 Interfaces . . . 18

6 Hardware Bench Prototype **19**
6.1 Hardware and connections . . . 19
6.2 Attitude estimation and virtual stabiliser . . . 19
6.3 One-ToF landing-readiness rules . . . 21
6.4 Latched gear sequence . . . 22
6.5 Firmware and desktop tools . . . 22
6.6 Demonstration checks . . . 23

7 Verification and Validation of the Simulation **24**
7.1 Synthetic testbeds . . . 24
7.2 Scenario verification . . . 25
7.3 Acceptance criteria and results . . . 26
7.4 Offline checks . . . 26
7.5 Inference latency . . . 28

8 Discussion **29**

9 Conclusion **30**

1 Introduction

Small fixed-wing and hybrid UAVs frequently land in unprepared environments (field strips, slopes, grass and debris) where the ground cannot be assumed to be a smooth, level runway. A conventional retractable undercarriage deploys on a mission command and has no knowledge of the surface it is about to meet. An undercarriage that is aware of the terrain could instead withhold deployment and trigger a go-around or a hover when the surface is unsafe.

1.1 Problem statement

Given a UAV in terminal descent, the system must (i) estimate the local terrain slope and surface character from low-cost range sensors, (ii) classify the terrain as safe or unsafe within a bounded, deterministic time, (iii) keep the aircraft attitude close to a reference so that the downward-looking sensors point along the vertical, and (iv) ensure that landing gear is never deployed while the terrain is classified as unsafe. The cost of the two possible classification errors is asymmetric: declaring unsafe terrain safe (a *false-safe* decision) can damage the aircraft, whereas the opposite error merely costs a missed landing opportunity. The design therefore targets a false-safe rate of zero.

1.2 Objectives and contributions

The work is organised around the following objectives.

1. A three-domain architecture (sensing, intelligence, control) in which perception interacts with the control loop only through a verdict and a boolean abort flag (Section 2).
2. A sensing scheme that derives terrain slope and local height variation from a triangular ToF array and adds an optical-acoustic comparison as a further cue (Sections 3 and 4.3 to 4.5).
3. A discrete complementary filter and an anti-windup PID loop for attitude estimation and pitch stabilisation, with an analysis of their dynamic and noise properties (Sections 4.1 and 4.2).
4. A learned terrain classifier, a depth-limited decision tree compiled into allocation-free C++ (Section 4.6).
5. A safety interlock with a rate-limited gear trajectory in which the abort flag has priority over the deploy request (Section 4.7), and an approach supervisor that sequences a simulated descent (Section 4.8).
6. A ROS 2 and Gazebo software-in-the-loop (SITL) implementation with four synthetic worlds and an automated verification harness (Sections 5 and 7).
7. A physical bench prototype on an Arduino Uno that exercises attitude estimation, a virtual stabiliser command, landing-readiness rules and a latched gear sequence on real IMU and ToF data (Section 6).

The source code of both prototypes is available in the project repository [1]. Throughout the report, the *implemented* behaviour of the code is distinguished from design intent, and claims are limited to what the evidence supports.

1.3 Organisation of the report

Section 2 introduces the architecture. Section 3 describes the hardware concept and its simulated counterpart. Section 4 presents and analyses the algorithms. Section 5 summarises the software implementation of the simulation, Section 6 describes the hardware bench prototype, Section 7 covers verification and validation, and Section 8 discusses limitations and future work. Section 9 concludes.

2 System Architecture

The system follows a decoupled *three-domain* model. Each domain has a single responsibility, a nominal update rate and a narrow, explicit interface (Figure 1). The separation is inspired by the partitioning ideas of integrated modular avionics (e.g. ARINC 653 [2]); it is an architectural decoupling by message passing, and no partition scheduling or worst-case execution-time analysis is claimed.

- The **sensing domain** acquires the ToF and sonar ranges, derives the terrain slope and range-variance features and publishes a seven-element feature vector at a nominal 50 Hz.
- The **intelligence domain** evaluates a compiled classifier on every feature vector and publishes a discrete verdict together with a boolean abort flag.
- The **control domain** runs the attitude estimator and the pitch stabiliser at the 100 Hz IMU rate, the gear controller at 50 Hz, and the approach supervisor at a nominal 50 Hz.

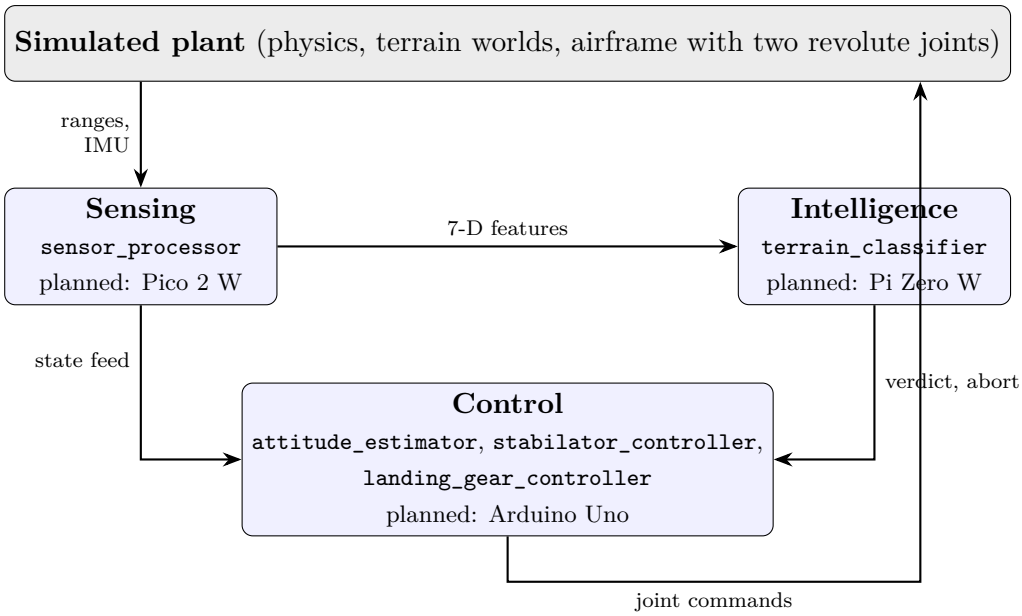


Figure 1: Three-domain architecture. Each domain is a set of ROS 2 nodes in the simulation; the boards named are the original deployment plan.

Realisations. The architecture has two realisations in this work. In the simulation, every domain is a set of ROS 2 nodes on a host computer; the boards named in Figure 1 are the original deployment plan and were not used. In the bench prototype of Section 6, sensing, the readiness decision and the control commands share one Arduino Uno and no classifier is used. There is no hardware-in-the-loop test.

Decoupling. The classifier's output reaches the control domain through a single boolean signal, and the attitude loop is driven by the IMU stream alone. A slow or failed perception node therefore cannot block the stabiliser, although it can leave the abort flag stale.

Fail-safe interlock. When the abort flag is asserted, the gear controller forces the deployment target to the stowed angle irrespective of the request (Section 4.7).

Zero-allocation inference. The trained classifier is compiled into static C++ comparisons without `malloc` or `new`, which gives a data-independent bound on the number of operations per decision (Section 4.6).

3 Hardware

3.1 Embedded computing targets

The original design assigns one embedded platform to each domain (Table 1), following the workload of each: the sensing domain needs bus access and floating-point arithmetic at moderate rates, the intelligence domain only comparisons, and the control domain predictable timing on a simple processor. In this work the simulation runs all domains on a host computer, and the bench prototype uses a single Uno (Section 6).

3.2 Sensors

Three sensing modalities are used (Table 2). Their complementary characteristics are the basis of the perception pipeline.

Domain	Planned platform	Nominal characteristics	Task
Sensing	Raspberry Pi Pico 2 W	RP2350 dual-core Arm microcontroller, up to 150 MHz, 520 kB SRAM [14]	Range acquisition and feature extraction
Intelligence	Raspberry Pi Zero W	Single-core 1 GHz ARM11 application processor, 512 MB RAM	Decision-tree inference, verdict and abort
Control	Arduino Uno	ATmega328P 8-bit AVR, 16 MHz, 2 kB SRAM [15]	Attitude filter, pitch PID, gear actuation

Table 1: Original embedded plan of the three domains and nominal manufacturer characteristics. None of these boards runs the simulated pipeline.

Function	Part	Design characteristics	Simulated model
Optical ranging ($\times 3$)	VL53L0X laser ToF [8]	Narrow beam, $\approx 25°$ field of view	3×3 ray fan over ± 0.04 rad, 50 Hz, range 0.03 to 2.5 m, 1 mm resolution, no noise
Acoustic ranging	HC-SR04 ultrasonic [12]	Wide conical beam, 30° to 40°	5×5 ray fan over $\pm 15°$, 20 Hz, range 0.02 to 4.0 m, 1 cm resolution
Attitude	MPU-6500 / MPU-9250 class IMU [9, 13]	Three-axis accelerometer and gyroscope	IMU at the airframe, 100 Hz; Gaussian noise $\sigma_\omega = 10^{-3}$ rad/s, $\sigma_a = 10^{-2}$ m/s^2

Table 2: Sensors of the system and their simulated models (values from the rig description of the repository).

Time-of-flight ranging. A ToF sensor measures the round-trip time Δt of an emitted light pulse and returns $d = c\,\Delta t/2$, with c the speed of light. The narrow beam gives a nearly point-like range measurement with millimetre resolution.

Ultrasonic ranging. The ultrasonic sensor emits a 40 kHz burst and measures the time until the first significant echo, $d = c_s\,\Delta t/2$, where the speed of sound depends on temperature as $c_s \approx 331.3 + 0.606\,T$ m/s (T in °C). Because the acoustic wavefront is wide, a physical sensor reports the nearest reflecting surface in its footprint. The simulated sonar is a ray fan, and the processing node takes the mean and variance of its valid rays (Section 4.4).

Inertial measurement. The accelerometer observes the specific force, which for a quasi-static vehicle is the reaction to gravity and therefore encodes pitch and roll; the gyroscope observes angular rate and is accurate in the short term but drifts when integrated.

Sensor geometry. The three ToF sensors form an isosceles triangle on the ventral plate, 0.070 m below the airframe origin: one fore sensor at (0.195, 0) m and a port and a starboard sensor at (0.125, $\pm$0.039) m, with the sonar at (0.15, 0) m (Figure 2). The fore sensor is therefore 0.070 m ahead of the line joining the other two, and the lateral baseline is 0.078 m; these two baselines are the constants used by the slope computation of Section 4.3.

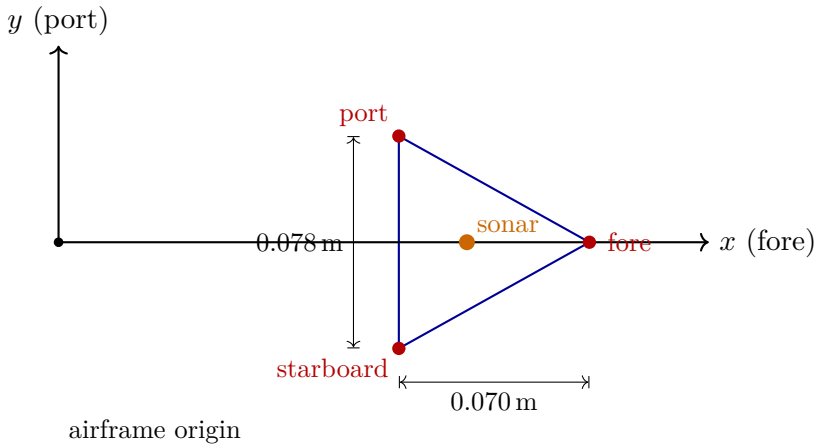


Figure 2: Ventral sensor plate (top view): ToF sensors at $(0.195, 0)$, $(0.125, 0.039)$ and $(0.125, -0.039)$ m and sonar at $(0.15, 0)$ m. The fore-aft baseline is 0.070 m and the lateral baseline 0.078 m.

3.3 Actuators

Two actuators are used, both realised as revolute joints (Table 3). The physical concept uses SG90 hobby servos [16]. In the simulation each joint is driven by a joint-position controller, which forms the inner loop of a cascade whose outer loop is the attitude or gear command of the control domain.

Joint	Range	Joint limits	Inner-loop gains
Stabilator (T-tail)	±0.4363 rad (±25°)	effort 1.2, speed 6.0 rad/s	K_p=30, K_i=0.2, K_d=1.5
Main landing gear	0 to 1.5708 rad (0° to 90°)	effort 2.0, speed 3.5 rad/s	K_p=25, K_i=0.2, K_d=1.0

Table 3: Actuated joints of the simulated rig. The inner-loop gains belong to the simulator's joint controllers and are distinct from the outer pitch-stabilisation loop of Section 4.2.

3.4 Simulated airframe

The airframe is modelled as a small fixed-wing aircraft: a cylindrical fuselage of 0.70 m length and 1.60 kg mass, a high wing of 0.90 m span, a T-tail carrying a 0.36 m horizontal stabilator (0.05 kg), and a retractable two-wheel main gear (0.11 kg). The sensor bay is on the ventral side of the fuselage and carries the ToF array and the sonar; the IMU is placed at the centre of gravity. The simulation runs with a 1 ms physics step. In the three static testbeds the airframe is attached to a fixed test stand 0.70 m above the ground; in the approach scenario of Section 4.8 it is carried along a flight rail. In neither case does the stabilator deflection change the motion of the body. The simulated inertial unit provides accelerations and angular rates only, as does the MPU6500 of the bench prototype.

4 Algorithms and Methods

This section develops the algorithms of the three domains. Throughout, θ denotes pitch, ϕ roll, $\boldsymbol{a} = [a_x, a_y, a_z]^\top$ the accelerometer output, $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^\top$ the gyroscope output and Δt the sampling interval.

4.1 Attitude estimation by complementary filtering

Accelerometer attitude. For a quasi-static vehicle with a z-axis measuring $+g$ when level, the accelerometer reads

$$\boldsymbol{a} = \begin{bmatrix} -g\sin\theta \\ g\cos\theta\sin\phi \\ g\cos\theta\cos\phi \end{bmatrix}, \tag{1}$$

from which the gravity-referenced angles follow as

$$\theta_{\mathrm{acc}} = \operatorname{atan2}\Big(-a_x,\ \sqrt{a_y^2 + a_z^2}\Big), \qquad \phi_{\mathrm{acc}} = \operatorname{atan2}(a_y,\ a_z). \tag{2}$$

These estimates are drift-free but corrupted by vehicle acceleration and vibration, because the accelerometer cannot distinguish gravity from linear acceleration.

Overflow-safe evaluation. On an 8-bit AVR target an `int` is 16 bits wide. With a $\pm 2\,g$ range the raw accelerometer reading is of the order of 16 384 counts per g, and its square ($\approx 2.7 \times 10^8$) overflows a signed 16-bit integer, giving a negative radicand and a `NaN` result. The squares in (2) are therefore evaluated after conversion to single-precision floating point.

Gyroscope integration. The angular rates integrate to attitude increments

$$\Delta\theta_{\mathrm{gyro}} = \omega_y\,\Delta t, \qquad \Delta\phi_{\mathrm{gyro}} = \omega_x\,\Delta t, \tag{3}$$

which are accurate over short intervals but accumulate sensor bias b_g as a linearly growing error.

Fusion. The two sources have complementary spectra: the gyroscope is reliable at high frequency, the accelerometer at low frequency. The discrete complementary filter blends them with a factor $\alpha \in (0, 1)$:

$$\theta_k = \alpha\big(\theta_{k-1} + \omega_{y,k}\Delta t\big) + (1-\alpha)\,\theta_{\mathrm{acc},k}, \qquad \phi_k = \alpha\big(\phi_{k-1} + \omega_{x,k}\Delta t\big) + (1-\alpha)\,\phi_{\mathrm{acc},k}, \tag{4}$$

with $\alpha = 0.90$ and $\Delta t = 0.01\,\mathrm{s}$ (100 Hz). Yaw is not estimated, because gravity carries no yaw information and the node publishes zero for it.

Frequency-domain interpretation. Taking the z-transform of (4) for the pitch channel gives

$$\hat{\theta}(z) = \underbrace{\frac{1-\alpha}{1-\alpha z^{-1}}}_{G_{\mathrm{LP}}(z)}\,\theta_{\mathrm{acc}}(z) + \underbrace{\frac{\alpha\,\Delta t}{1-\alpha z^{-1}}}\,\omega_y(z). \tag{5}$$

The accelerometer path is a first-order low-pass filter with unit DC gain. If the gyroscope signal is the backward difference of the true angle, $\omega_y = (1 - z^{-1})\theta/\Delta t$, its path becomes $\alpha(1 - z^{-1})/(1 - \alpha z^{-1})$, a high-pass filter, and the two paths sum to

$$\frac{(1-\alpha) + \alpha(1 - z^{-1})}{1 - \alpha z^{-1}} = \frac{1 - \alpha z^{-1}}{1 - \alpha z^{-1}} = 1,$$

so the filter is exactly complementary: it passes the true angle without distortion and only filters the *errors* of the two sensors. The pole at $z = \alpha$ corresponds to a time constant and corner frequency

$$\tau = -\frac{\Delta t}{\ln\alpha} \approx 0.095\ \mathrm{s}, \qquad f_c = \frac{1}{2\pi\tau} \approx 1.7\ \mathrm{Hz}. \tag{6}$$

Attitude changes slower than about 1.7 Hz are therefore taken from the accelerometer, faster ones from the gyroscope.

Error analysis. Two error sources are of interest.

1. *Gyroscope bias.* For a constant bias b_g and a constant true angle, the steady state of (4) is $\hat{\theta} - \theta = \alpha \Delta t \, b_g/(1-\alpha) = 0.09\,\mathrm{s} \cdot b_g$. A bias of 0.01 rad/s, for example, produces a bounded error of 9×10^{-4} rad (0.05°) instead of an unbounded drift.
2. *Accelerometer noise.* For small angles, $\delta\theta_{\mathrm{acc}} \approx \delta a_x/g$, so the simulated noise $\sigma_a = 0.01\,\mathrm{m/s^2}$ gives $\approx 1.0 \times 10^{-3}$ rad per sample. The low-pass path reduces the standard deviation by $\sqrt{(1-\alpha)/(1+\alpha)} \approx 0.23$, leaving $\approx 2.3 \times 10^{-4}$ rad (0.013°).

Algorithm 1 Complementary filter update (executed at 100 Hz)

Input: raw a_x, a_y, a_z, $\omega_x, \omega_y, \omega_z$, previous state $(\theta_{k-1}, \phi_{k-1})$, α, Δt
Output: filtered pitch θ_k and roll ϕ_k

convert a_x, a_y, a_z to single-precision floating point
$\theta_{\mathrm{acc}} \leftarrow \mathrm{atan2}(-a_x, \sqrt{a_y^2 + a_z^2})$
$\phi_{\mathrm{acc}} \leftarrow \mathrm{atan2}(a_y, a_z)$
$\theta_k \leftarrow \alpha(\theta_{k-1} + \omega_y \Delta t) + (1-\alpha)\theta_{\mathrm{acc}}$
$\phi_k \leftarrow \alpha(\phi_{k-1} + \omega_x \Delta t) + (1-\alpha)\phi_{\mathrm{acc}}$
publish (ϕ_k, θ_k) as the attitude estimate

4.2 Active pitch stabilisation

Purpose. Terrain geometry is measured in the sensor frame; if the fuselage pitches during descent, the three ToF beams no longer point along the local vertical and the apparent slope is biased. The horizontal stabilator is therefore driven by a closed-loop controller that regulates the estimated pitch to a setpoint θ_{ref} (default 0 rad). Deflecting the tail surface changes the tail lift and hence the pitching moment about the centre of gravity, which is the actuation path exploited by the controller. In the simulation this path is not closed physically, because the airframe is held by a test stand or a flight rail; the loop acts on the estimated pitch and drives the stabilator joint.

Control law. With the error $e_\theta(t) = \theta_{\mathrm{ref}} - \hat{\theta}(t)$ the continuous PID law is

$$u(t) = K_p \, e_\theta(t) + K_i \int_0^t e_\theta(\tau)\, d\tau + K_d \, \frac{de_\theta(t)}{dt}. \tag{7}$$

The proportional term provides the primary restoring action, the integral term removes steady-state error caused by constant disturbances such as trim offsets, and the derivative term adds damping and limits overshoot.

Discrete implementation and anti-windup. At 100 Hz the loop is realised as

$$S_k = \mathrm{clamp}\big(S_{k-1} + e_k \, \Delta t,\ -S_{\max},\ S_{\max}\big), \tag{8}$$

$$D_k = K_d \, \frac{e_k - e_{k-1}}{\Delta t}, \tag{9}$$

$$u_k = \mathrm{sat}\big(K_p \, e_k + K_i \, S_k + D_k,\ -u_{\max},\ u_{\max}\big), \qquad u_{\max} = 0.4363 \text{ rad}, \tag{10}$$

The command is saturated at the mechanical limit of the stabilator. Without further measures, an integrator would continue to accumulate while the actuator is saturated (for example during a strong gust) and cause a large overshoot when the error finally changes sign. Clamping the integral state $S_k = \int e \, dt$ to $[-S_{\max}, S_{\max}]$ prevents this *integrator windup* [5]. The loop is executed for every attitude sample, with the derivative taken on the error and a fall-back step

of 10 ms if the measured step is non-positive or longer than 0.2 s. The gains are $K_p = 1.8$, $K_i = 0.05$ and $K_d = 0.25$, with $S_{\max} = 0.20$ rad s; they are run-time parameters of the node. The resulting command is sent to the simulated stabilator joint, whose own position controller (Table 3) acts as the fast inner loop of the cascade.

Algorithm 2 Discrete PID pitch stabiliser with anti-windup (100 Hz)

Input: estimated pitch $\hat{\theta}$, setpoint θ_{ref}, gains K_p, K_i, K_d, limits $S_{\max}, u_{\max}$, state (S_{k-1}, e_{k-1})
Output: stabilator command u_k

$e_k \leftarrow \theta_{\text{ref}} - \hat{\theta}$
$S_k \leftarrow \text{clamp}(S_{k-1} + e_k \Delta t,\ -S_{\max},\ S_{\max})$
$D_k \leftarrow K_d\,(e_k - e_{k-1})/\Delta t$
$u_k \leftarrow \text{clamp}(K_p e_k + K_i S_k + D_k,\ -u_{\max},\ u_{\max})$
store (S_k, e_k) and publish u_k to the stabilator

4.3 Terrain slope from the ToF triangle

Range extraction. Each ToF sensor delivers a scan of nine rays. The processing node keeps, for every sensor, the minimum range among the rays that are finite and inside the sensor limits, and holds the most recent value; if a scan contains no valid ray, the value 0.85 m is used. Denote the three values d_f (fore), d_p (port) and d_s (starboard). There is no averaging window and no freshness check: the features are computed from the latest retained scans.

Planar model. With the sensor plate level and the beams vertical, a locally planar surface with gradients g_x (along the fore axis) and g_y (along the port axis) produces ranges

$$d_i = d_0 - \big(g_x\, x_i + g_y\, y_i\big), \qquad i \in \{f, p, s\}, \tag{11}$$

where d_0 is the range at the origin of the plate and (x_i, y_i) are the sensor positions of Figure 2. Two differences isolate the gradients. The fore sensor and the midpoint of the other two are separated by $\Delta x = 0.070$ m along x and have the same y, and the port and starboard sensors are separated by $\Delta y = 0.078$ m along y at the same x. Hence

$$d_f - \tfrac{1}{2}(d_p + d_s) = -g_x\,\Delta x, \qquad d_p - d_s = -g_y\,\Delta y. \tag{12}$$

Implemented estimate. The sensing node converts the two differences into angles,

$$\beta_x = \text{atan2}\big(d_f - \tfrac{1}{2}(d_p + d_s),\ 0.070\big), \qquad \beta_y = \text{atan2}\big(d_p - d_s,\ 0.078\big), \qquad \beta = \sqrt{\beta_x^2 + \beta_y^2}. \tag{13}$$

For a plane inclined along one axis only, $|\beta_x| = \arctan|g_x|$ is the exact inclination of the surface, because the range difference divided by the baseline equals the rise over the run. For a plane inclined along both axes, β approximates the true inclination $\arctan\sqrt{g_x^2 + g_y^2}$ by a root-sum-square of the per-axis angles; for $g_x = g_y = 0.2$ the two values are 0.279 and 0.276 rad. The sign of β_x and β_y depends on whether the ground rises or falls along the axis, and only the magnitude $\beta \geq 0$ (and $|\beta_y|$) is used downstream. The computation uses the two baseline constants directly rather than a surface normal from the cross product of the span vectors, and it does not use the IMU.

Consequences of the assumptions. The estimate is valid only if the plate is level. If the aircraft is pitched by θ, the beams are no longer vertical and β_x is shifted by roughly θ; since the runtime does not subtract the filtered attitude, body pitch appears as terrain slope. This is the

reason for stabilising pitch (Section 4.2), but in the simulation the airframe is held by a stand or a rail, so the coupling is never exercised. The estimate also treats the 0.070 m by 0.078 m patch as planar, so it measures the surface at a scale of about 0.1 m and cannot see slopes of larger extent or smaller obstacles.

Noise sensitivity. The short baselines amplify range noise. For independent range errors of standard deviation σ, $\mathrm{Var}\big(d_f - \frac{1}{2}(d_p + d_s)\big) = \frac{3}{2}\sigma^2$ and $\mathrm{Var}(d_p - d_s) = 2\sigma^2$, so for small angles

$$\sigma_{\beta_x} \approx \frac{\sqrt{1.5}\,\sigma}{0.070} \approx 17.5\,\sigma \text{ rad/m}, \qquad \sigma_{\beta_y} \approx \frac{\sqrt{2}\,\sigma}{0.078} \approx 18.1\,\sigma \text{ rad/m}. \tag{14}$$

A range noise of 2 mm, which is of the order expected for a real ToF module, would give about 0.035 rad (2°) per axis. The simulated ToF sensors are noise-free with 1 mm resolution, so this noise is not present in the simulation results.

4.4 Optical-acoustic comparison

The ToF beams are narrow and the acoustic fan is wide, so the two sensor types sample the surface at different scales. The sensing node computes two quantities that compare them.

Mean difference. The optical altitude is the mean of the three ToF ranges, $\bar{d}_{\mathrm{ToF}} = \frac{1}{3}(d_f + d_p + d_s)$, and the acoustic range $\bar{d}_{\mathrm{sonar}}$ is the mean of the valid rays of the latest sonar scan. Their difference is

$$\Delta d = \bar{d}_{\mathrm{sonar}} - \bar{d}_{\mathrm{ToF}}. \tag{15}$$

The oblique rays of the ±15° fan travel farther than a vertical ray, which gives Δd a positive offset on flat ground, and surface relief inside the acoustic footprint changes it further. In the three static worlds Δd lies between 0.010 and 0.033 m (Table 18). The original concept used this difference as a cue for penetrable vegetation, where the optical and acoustic channels may see different surfaces. The simulated canopy is a solid body, no optical penetration is modelled, the sign of Δd in the synthetic canopy is not consistently positive, and no real-material experiment was made. The cue is therefore computed but is not validated as a vegetation detector.

Scan variance. The second quantity is the population variance of the valid sonar rays within one scan,

$$\sigma^2_{\mathrm{sonar}} = \frac{1}{n}\sum_{j=1}^{n}\big(r_j - \bar{d}_{\mathrm{sonar}}\big)^2, \qquad \sigma^2_{\mathrm{sonar}} \geq 10^{-5}\ \mathrm{m}^2, \tag{16}$$

which measures the relief of the surface across the acoustic cone at a single instant. It is not a temporal variance.

4.5 Feature extraction

Spatial variance of the ToF ranges. The local height-variation measure is the population variance of the three ToF ranges,

$$\sigma^2_{\mathrm{sp}} = \frac{1}{3}\sum_{i\in\{f,p,s\}}\big(d_i - \bar{d}_{\mathrm{ToF}}\big)^2, \qquad \sigma^2_{\mathrm{sp}} \geq 10^{-5}\ \mathrm{m}^2. \tag{17}$$

For a plane (11) the range deviations are $-g_x(x_i - \bar{x}) - g_y(y_i - \bar{y})$. With the plate geometry of Figure 2, the variance of the x coordinates is $\frac{1}{3}\sum(x_i - \bar{x})^2 = 1.089 \times 10^{-3}\,\mathrm{m}^2$ and that of the y coordinates is $1.014 \times 10^{-3}\,\mathrm{m}^2$, and the cross term vanishes because the sensors are symmetric about $y = 0$. Therefore

$$\sigma^2_{\mathrm{sp}} = 1.089 \times 10^{-3}\,g_x^2 + 1.014 \times 10^{-3}\,g_y^2 \ \ \mathrm{m}^2. \tag{18}$$

Under the level-plate and vertical-beam assumptions, three range-derived points at non-collinear sensor positions always define a plane. The gradients inferred from those three values therefore satisfy (18) algebraically, even when the underlying surface is curved or contains a step. This variance measures local height variation and does not independently establish roughness or non-planarity. Detecting departures from a fitted plane requires redundant spatial measurements, for example at least four independent points, or additional scans with known geometry. For the Beta ramp the apparent slope of 0.348 rad gives $g_x = \tan 0.348 = 0.363$ and a variance of $1.4 \times 10^{-4}\,\mathrm{m}^2$, consistent with the measured $1.43 \times 10^{-4}\,\mathrm{m}^2$ (Table 18). This agreement is a consistency check on the calculation rather than independent evidence of surface planarity.

Feature vector. The node publishes the vector of Table 4 on a 20 ms timer (nominal 50 Hz) and the altitude $\bar{d}_{\mathrm{ToF}}$ on a separate topic. The column names of the recorded dataset (Section 4.6) predate the current code, and their meaning differs for three entries: the fourth feature is a variance within one sonar scan rather than a temporal variance of Δd, the sixth is the magnitude of the roll slope and not the gyroscope norm, and the seventh repeats the total slope and is not a filtered attitude error. Table 4 lists what the runtime computes.

Index	Quantity	Definition	Unit	Used
0	Mean ToF range	$\bar{d}_{\mathrm{ToF}}$	m	no
1	Spatial variance	σ^2_{sp}, (17)	m^2	yes
2	Optical-acoustic difference	$\Delta d = \bar{d}_{\mathrm{sonar}} - \bar{d}_{\mathrm{ToF}}$	m	no
3	Sonar scan variance	$\sigma^2_{\mathrm{sonar}}$	m^2	no
4	Total slope	β, (13)	rad	no
5	Roll slope magnitude	$\lvert\beta_y\rvert$	rad	no
6	Total slope (repeated)	β, (13)	rad	yes

Table 4: Feature vector computed by the sensing node. The last column states whether the trained tree reads the entry.

Algorithm 3 Feature extraction cycle of the sensing node (every 20 ms)

Input: latest ToF scans (d_f, d_p, d_s) (minimum valid ray, fallback 0.85 m), latest sonar statistics $(\bar{d}_{\mathrm{sonar}}, \sigma^2_{\mathrm{sonar}})$
Output: feature vector $\boldsymbol{x} \in \mathbb{R}^7$ and altitude $\bar{d}_{\mathrm{ToF}}$

$\bar{d}_{\mathrm{ToF}} \leftarrow (d_f + d_p + d_s)/3$
$\sigma^2_{\mathrm{sp}} \leftarrow \max\big(10^{-5},\ \frac{1}{3}\sum_i (d_i - \bar{d}_{\mathrm{ToF}})^2\big)$
$\Delta d \leftarrow \bar{d}_{\mathrm{sonar}} - \bar{d}_{\mathrm{ToF}}$
$\beta_x \leftarrow \mathrm{atan2}(d_f - \frac{1}{2}(d_p + d_s),\ 0.070)$; $\quad \beta_y \leftarrow \mathrm{atan2}(d_p - d_s,\ 0.078)$
$\beta \leftarrow \sqrt{\beta_x^2 + \beta_y^2}$
$\boldsymbol{x} \leftarrow [\bar{d}_{\mathrm{ToF}},\ \sigma^2_{\mathrm{sp}},\ \Delta d,\ \sigma^2_{\mathrm{sonar}},\ \beta,\ |\beta_y|,\ \beta]$
publish $\boldsymbol{x}$ and $\bar{d}_{\mathrm{ToF}}$

4.6 Edge-AI terrain classifier

Task and labels. The classifier maps the feature vector $\boldsymbol{x} \in \mathbb{R}^7$ to a class $y \in \{0, 1, 2\}$. The labels describe the suitability of the terrain below the aircraft for landing (Table 5). Class 0 (SAFE) denotes a level, rigid surface on which the gear may be deployed. The two unsafe classes distinguish the *reason* for rejecting the terrain: class 1 (UNSAFE_SLOPE) denotes a geometric

hazard, an inclined surface that risks rollover, whereas class 2 (UNSAFE_VEGETATION) denotes an irregular surface of mounds and a canopy body, where the footing is uncertain. Both unsafe classes have the same operational consequence: the abort flag is asserted for every verdict other than SAFE, which inhibits gear deployment (Section 4.7).

Label	Testbed	Terrain	Intended signature
0 SAFE	Alpha	Flat ground plane	Near-zero slope and range variance
1 UNSAFE_SLOPE	Beta	20° inclined ramp and a stepped curb block	Large slope, small range variance on the planar ramp
2 UNSAFE VEGETATION	Gamma	Three box-shaped mounds (0.08 to 0.14 m high) and a central solid cylinder (0.18 m radius, 0.16 m high)	Large slope and range variance within the sensor patch

Table 5: Class labels, the testbed that produces each class and the intended signature.

Dataset. A recording node subscribes to the feature topic, keeps the first seven values of each message together with the label of the selected world, and writes one row per message to a comma-separated file; 500 samples per world is the default, so one run in each testbed gives 1 500 labelled vectors (500 per class). Two properties of the data matter for the interpretation of the results. First, the labels are properties of the *world*: every row inherits the label of its testbed irrespective of the instantaneous sensor reading. Second, the rig is static, the sensors are noise-free and the geometry is fixed, so the recorded vectors of one world are almost identical (Table 6). Most columns have zero standard deviation within a class: only the angular-rate column and the last column vary, with standard deviations of about 6×10^{-4} and 2×10^{-4}, respectively. The dataset therefore consists of three tight clusters rather than three distributions of terrain.

Column (recorded name)	Class 0	Class 1	Class 2
`mean_tof` (m)	1.640000	1.440778	1.506667
`spatial_variance` (m^2)	0	0.000144	0.001422
`delta_d` (m)	0.057811	0.051345	0.191144
`delta_d_variance`	0	0	0
`beta_slope` (rad)	0	0.275867	0.726643
`omega_norm` (rad/s)	0.001647	0.001597	0.001550
`tilt_error` (rad), mean	0.000238	0.275854	0.726638
`tilt_error`, minimum	0.000000	0.274958	0.726070
`tilt_error`, maximum	0.000753	0.276555	0.727235

Table 6: Class means of the recorded dataset (500 samples per class). The standard deviation within a class is zero for every row except `omega_norm` (about 6×10^{-4}) and `tilt_error` (about 2×10^{-4}).

The dataset was recorded with an earlier version of the sensing node. Its values differ from the live runtime values of Table 18 (for example the mean range of Beta is 1.44 m in the dataset and 0.53 m in the verification run, and the slope 0.276 against 0.348 rad), and the repository advises recording the actual feature definitions alongside any new dataset. The tree was not retrained for the current runtime; its thresholds are applied to the live features, as discussed below.

Model choice. A classification tree (CART [6]) was selected. An initial concept considered a small multilayer perceptron with quantisation for microcontrollers. A tree offers several advan-

tages for this application: (i) its decision path is a short sequence of threshold comparisons that can be inspected and audited, which matters for a safety function; (ii) it needs no matrix arithmetic or activation functions; (iii) it compiles into code with a constant bound on the operations and no dynamic allocation; and (iv) the features are physically meaningful, so that axis-aligned splits are natural.

Induction. A tree is grown greedily. For a node holding the sample set S with class proportions p_k, the Gini impurity is

$$G(S) = 1 - \sum_{k=0}^{2} p_k^2, \tag{19}$$

which is 0 for a pure node and 2/3 for three equally frequent classes. For every feature j and candidate threshold t (the midpoints between consecutive sorted feature values) the set is split into $S_L = \{\boldsymbol{x} : x_j \le t\}$ and $S_R = S \setminus S_L$, and the split with the largest impurity decrease is chosen,

$$(j^*, t^*) = \arg\max_{j,t} \Big[G(S) - \tfrac{|S_L|}{|S|} G(S_L) - \tfrac{|S_R|}{|S|} G(S_R) \Big]. \tag{20}$$

The procedure recurses on S_L and S_R until a node is pure or the maximum depth is reached; a leaf predicts the majority class of its samples (Algorithm 4). A depth limit of four bounds the model to at most 16 leaves and 15 decision nodes and caps inference at four comparisons per decision.

Algorithm 4 Greedy induction of a depth-limited classification tree

Input: labelled set S, current depth ℓ, maximum depth $\ell_{\max} = 4$
Output: tree node

function GROW(S, ℓ)
  if $G(S) = 0$ **or** $\ell = \ell_{\max}$ **then**
    return Leaf(majority class of S)
  end if
  $(j^*, t^*) \leftarrow \arg\max_{j,t}$ impurity decrease of (20)
  $S_L \leftarrow \{\boldsymbol{x} \in S : x_{j^*} \le t^*\}$; $S_R \leftarrow S \setminus S_L$
  return Node(j^*, t^*, GROW($S_L, \ell + 1$), GROW($S_R, \ell + 1$))
end function

Training configuration. The parameters of the training procedure are listed in Table 7. The dataset is split 75 %/25 % into 1 125 training and 375 test samples with stratification, and a fixed random seed makes the split and the tree reproducible. Training uses the scikit-learn implementation [7]; every parameter not listed takes the library default.

Learned model. Although the depth limit is four, the fitted tree is small (Figure 3): two decision nodes and three leaves, using two of the seven feature slots (indices 6 and 1). Growth stopped after two levels because every leaf was pure.

1. *Root split on feature 6.* Samples with a value of at most 0.137855 rad ($\approx 7.9°$) are classified SAFE. In the dataset, the largest class-0 value is 0.000753 and the smallest class-1 value is 0.274958, and $(0.000753 + 0.274958)/2 = 0.137855$, so the threshold is the midpoint of the gap between the flat-ground cluster and the ramp cluster. It is a property of the data gap and not a certified slope limit. In the current runtime, feature 6 is the total slope β of (13), so the node acts as a slope detector.

2. *Second split on the spatial variance.* Among the remaining samples, those with $\sigma_{\mathrm{sp}}^2 \le 0.000783\,\mathrm{m}^2$ are classified UNSAFE_SLOPE and the others UNSAFE_VEGETATION. Again

Parameter	Value
Algorithm	CART classification tree (`DecisionTreeClassifier`)
Inputs	7 recorded feature columns (Table 6); current runtime definitions differ (Table 4); single precision at inference
Classes	3 (SAFE, UNSAFE_SLOPE, UNSAFE_VEGETATION)
Samples	1 500 (1 125 training, 375 test)
Train/test split	75 %/25 %, stratified by class, random seed 42
Split criterion	Gini impurity, (20)
Maximum depth	4
Random seed of the tree	42
Other parameters	Library defaults: all seven features considered at every split, minimum 2 samples to split a node, minimum 1 sample per leaf, no class weights, no pruning

Table 7: Training configuration of the terrain classifier.

$(0.000144 + 0.001422)/2 = 0.000783$, the midpoint between the Beta and Gamma clusters. For planar returns inclined along the fore axis, (18) places this threshold at $g_x = \sqrt{0.000783/0.001089} = 0.85$, an inclination of about 40°. On such surfaces the second split is thus equivalent to a second slope threshold, and it separates the UNSAFE_VEGETATION class from the UNSAFE_SLOPE class by the magnitude of the height differences within the 0.1 m patch.

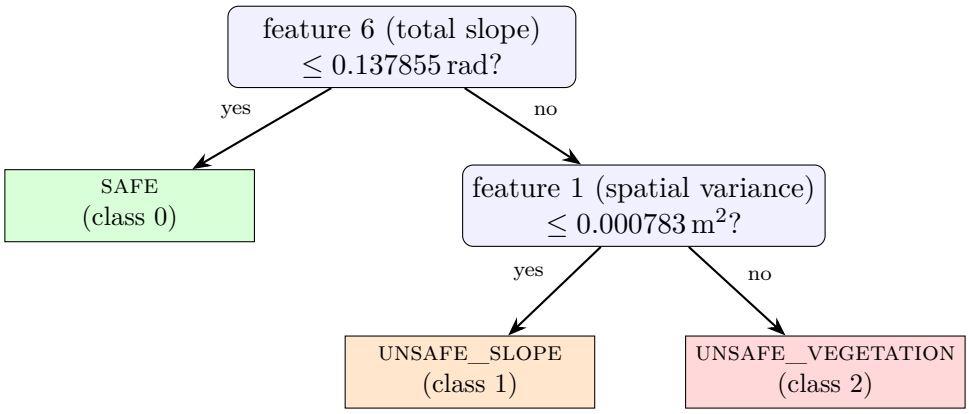


Figure 3: Trained classification tree: two decision nodes and three leaves, using two of the seven features.

The optical-acoustic features of Section 4.4 (indices 2 and 3) were not selected, because the two thresholds above already separate the clusters perfectly. These data therefore give no evidence that the optical-acoustic cue adds information.

Compilation to C++. After training, the tree is traversed recursively and each internal node is emitted as an `if/else` comparison of a feature against a single-precision threshold, each leaf as a `return` of the class index (Algorithm 5). The generated header contains a single function, `predict_terrain(const float*)`, whose structure is

```
inline int32_t predict_terrain(const float * features)
{
  if (features[j] <= t) {  // internal node
    ...                    // left subtree
  } else {
    ...                    // right subtree
```

```
  }                       // leaf: return <class index>;
}
```

It depends only on `<cstdint>`, performs no dynamic allocation and has a data-independent upper bound of four comparisons; the trained tree needs two. The thresholds are emitted with six decimal places in single precision.

Algorithm 5 Generation of the inference code from a trained tree

Input: trained tree, feature names
Output: C++ source of `predict_terrain`

```
function EMIT(node, depth)
    if node is internal then
        write "if (features[j] <= t) {"
        EMIT(left child, depth+1)
        write "} else {"
        EMIT(right child, depth+1)
        write "}"
    else
        write "return arg max_k (class counts of node)"
    end if
end function
```

Runtime behaviour. The classifier node evaluates every incoming feature vector and publishes the verdict and the abort flag once per accepted message, with the abort flag set for every verdict other than SAFE. Messages with fewer than seven elements are rejected. The verdict is not filtered: there is no debouncing or hysteresis, so the abort flag follows the instantaneous classification.

Safety-oriented metrics. Because the error costs are asymmetric, overall accuracy is not the decisive figure. Two dedicated metrics are used,

$$\mathrm{FSR} = \frac{|\{i : y_i \neq 0 \wedge \hat{y}_i = 0\}|}{|\{i : y_i \neq 0\}|}, \qquad R_{\mathrm{safe}} = \frac{|\{i : y_i = 0 \wedge \hat{y}_i = 0\}|}{|\{i : y_i = 0\}|}, \tag{21}$$

the false-safe rate (FSR) and the recall of the safe class, with targets $\mathrm{FSR} = 0$ and $R_{\mathrm{safe}} \geq 0.98$. The Gini criterion treats all misclassifications equally, so the safety requirement is enforced by evaluation rather than by the training objective.

Test-set results. The tree was evaluated on the 375 held-out samples, which contain 125 samples of each class. The confusion matrix is shown in Table 8 and Figure 4: all 375 samples were classified correctly, so accuracy, precision, recall and F1-score are all 1.00, $\mathrm{FSR} = 0/250 = 0$ and $R_{\mathrm{safe}} = 125/125 = 1$.

Actual \ predicted	SAFE	UNSAFE_SLOPE	UNSAFE_VEGETATION
SAFE	125	0	0
UNSAFE_SLOPE	0	125	0
UNSAFE_VEGETATION	0	0	125

Table 8: Confusion matrix on the 375-sample test set.

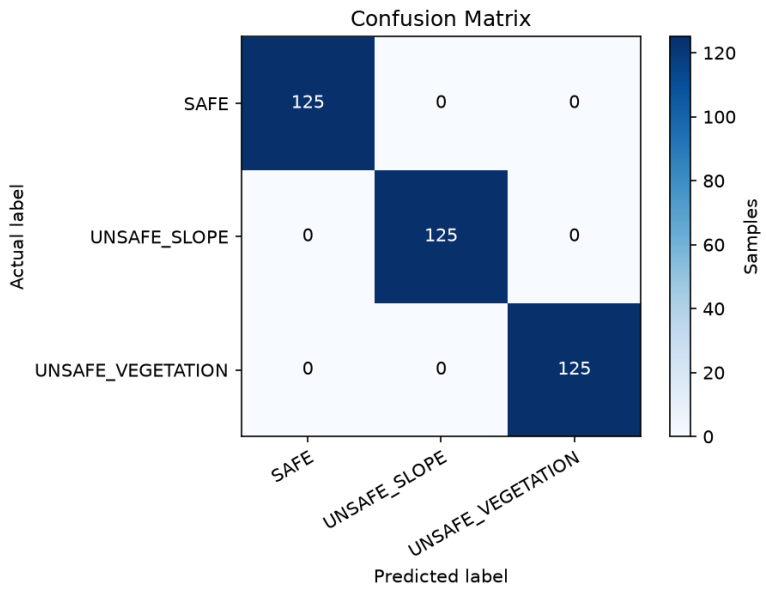


Figure 4: Confusion matrix of the classifier on the held-out test set.

This result must be read with the data in mind. Because the classes are separated by gaps of 0.27 rad and 0.0013 m^2, much larger than the spread within a class, and the thresholds are the midpoints of those gaps, any random split yields a perfect score. The test samples are in addition drawn from the same runs as the training samples and carry per-world labels. The score therefore shows that the three synthetic terrain types are separable, and it does not estimate performance on unseen terrain.

Execution time. Since a decision requires at most four comparisons (two in the trained tree), the latency is dominated by memory access and is of the order of tens of nanoseconds on the host (Section 7).

4.7 Gear control and safety interlock

Interlocked target. Let $D_k \in \{0, 1\}$ be the deploy request, published by the supervisor or by an operator, and $A_k \in \{0, 1\}$ the abort flag. The target gear angle is

$$q_k^* = \frac{\pi}{2}\, D_k\, (1 - A_k) \tag{22}$$

so that $q_k^* = 0$ (stowed) whenever $A_k = 1$, regardless of the request. The interlock thus has priority over the command. The controller contains no altitude logic of its own; the decision to request deployment belongs to the supervisor (Section 4.8) or the operator.

Rate limiting. To avoid abrupt mechanical loading and to bound the actuation speed, the target is passed through a rate limiter,

$$q_k = q_{k-1} + \mathrm{clamp}\big(q_k^* - q_{k-1},\ -\dot{q}_{\max}\Delta t,\ \dot{q}_{\max}\Delta t\big), \tag{23}$$

with $\dot{q}_{\max} = 0.50$ rad/s and $\Delta t = 0.02$ s (50 Hz), i.e. a maximum step of 0.01 rad. A full sweep takes $(\pi/2)/\dot{q}_{\max} \approx 3.14$ s. Because the same limiter acts when the abort asserts during deployment, the gear reverses at the permitted speed and reaches the stowed angle in at most 3.14 s. The published gear status is the controller's own trajectory q_k and not a measurement of the joint position.

Algorithm 6 Landing gear controller step with abort interlock (50 Hz)

Input: deploy request D, abort flag A, current command q_{k-1}, $\dot{q}_{\max}$, Δt
Output: gear command q_k (published) and gear status

if A **or not** D **then**
    $q^* \leftarrow 0$
else
    $q^* \leftarrow \pi/2$
end if
$\delta \leftarrow \mathrm{clamp}(q^* - q_{k-1},\ -\dot{q}_{\max}\Delta t,\ \dot{q}_{\max}\Delta t)$
$q_k \leftarrow q_{k-1} + \delta$
publish q_k as command and as status telemetry

4.8 Perception-driven approach supervisor

For a closed-loop approach scenario the simulation adds a flight supervisor that turns the classifier's output into a mission sequence. The supervisor is part of the repository's flight configuration, but the static verification of Section 7 does not exercise it and no flight results are claimed. The supervisor evaluates a five-state machine every 20 ms (Figure 5), using the ground clearance h from the sensing domain and the abort flag from the intelligence domain, and publishes the pitch setpoint for the stabiliser, the gear-deploy request, a touchdown flag and a state label. Its parameters are listed in Table 9.

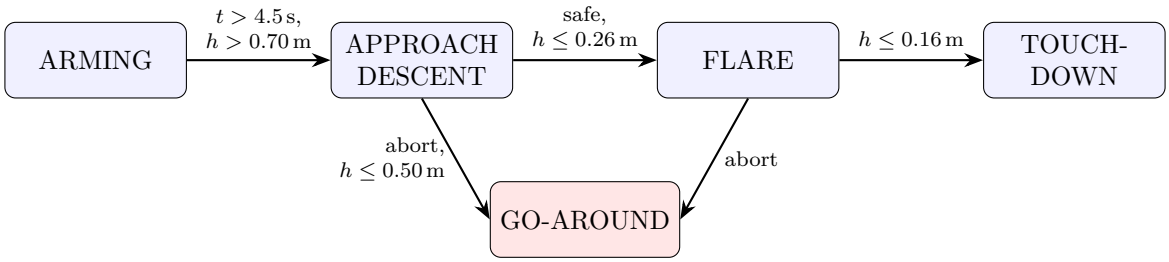


Figure 5: States of the approach supervisor. The arming variable t is a nominal callback-count timer, not elapsed simulation time. The go-around state has no exit transition in the implementation.

In ARMING the pitch setpoint is 0 and the gear is stowed. The machine arms only when its nominal callback-count timer exceeds 4.5 s and the vehicle is confirmed airborne ($h > 0.70\,\mathrm{m}$). The implementation uses a wall timer and increments this counter by 0.02 s on each callback, rather than measuring elapsed simulation time. Callback delays, slow simulation or a paused simulation can therefore make it diverge from the simulation clock. In APPROACH DESCENT the setpoint is a shallow descent attitude and the gear stays stowed above the decision height $H_{\mathrm{DH}} = 0.50\,\mathrm{m}$. At or below the decision height an abort leads immediately to a go-around, whereas a clear abort flag issues the deploy request and the approach continues; at the flare height the machine enters FLARE with a small nose-up setpoint. An abort during the flare also leads to a go-around. At the touchdown height the machine enters TOUCHDOWN, the setpoint returns to zero, the gear remains deployed and the touchdown flag stops the motion. In GO-AROUND the setpoint becomes a climb attitude and the gear is stowed. The interlock of Section 4.7 remains active in parallel: the gear controller forces the stowed angle whenever the abort flag is set.

Known interface fault. The attitude estimator publishes a stamped vector, whereas the supervisor subscribes with an unstamped vector type, so the supervisor does not receive the pitch estimate. The pitch value is not used in its state decisions, which depend on ground clearance,

Parameter	Value	Role
Descent pitch setpoint	-0.045 rad	APPROACH DESCENT
Flare pitch setpoint	$+0.050$ rad	FLARE
Climb pitch setpoint	$+0.200$ rad	GO-AROUND
Decision height H_{DH}	0.50 m	Abort check and gear deployment
Flare height	0.26 m	Start of FLARE
Touchdown height	0.16 m	Start of TOUCHDOWN
Arming conditions	$t > 4.5$ s (nominal callback counter), $h > 0.70$ m	Leaves ARMING

Table 9: Parameters of the approach supervisor (all except the arming conditions are ROS 2 parameters).

the abort flag and time, so the behaviour described here is unaffected, but the subscription is ineffective.

Approach testbed and carrier. The approach scenario uses a fourth world that places the three terrain types in sequence along a 180 m corridor: irregular vegetation mounds between $x = 5$ and 25 m, a 20° ramp (2.2 m long, centred at $x = 40$ m) followed by a 0.20 m stepped curb at $x = 48$ m, and a clear runway section from $x = 58$ to 110 m. The airframe is not flown by aerodynamic forces: it is mounted on two position-controlled prismatic joints (a flight rail), and a carrier script commands its motion. The carrier advances along x at 3.0 m/s, holds a clearance of 0.85 m up to $x = 25$ m, then descends linearly at 0.0115 m per metre travelled down to 0.16 m, climbs at 0.60 m/s up to 3 m when the pitch setpoint exceeds 0.10 rad (go-around), and stops at touchdown. The supervisor therefore decides, but the trajectory is prescribed, and the pitch loop of Section 4.2 acts on a vehicle whose body motion is not coupled to the stabilator. A simplified pitching-moment node exists in the source tree but is not launched.

5 Software Implementation

5.1 Platform

The system is implemented for Ubuntu 24.04 with ROS 2 Jazzy Jalisco [3] and Gazebo Harmonic [4]. The domain nodes are written in C++ and built with `colcon`; the data-collection, training and verification tools are Python scripts. A bridge maps Gazebo topics to ROS 2, and a launch file starts the simulator (with or without GUI), spawns the airframe in the selected world and starts all nodes with simulation time enabled. The static verification uses the launch option that disables the flight mode.

5.2 Nodes

5.3 Interfaces

The principal topics are listed in Table 11. Raw sensor streams and the sensing node's inputs use the sensor-data quality of service (best effort) to favour freshness over delivery guarantees; processed and control topics use reliable delivery. A queue depth of ten does not make a topic latched, so a late-joining subscriber does not receive earlier values of the deploy request or pitch setpoint. The coordinate frames and sensor poses are defined in the rig description of the repository.

Node	Domain	Function	Rate
`sensor_processor`	Sensing	Range extraction, slope, variances, features (Algorithm 3)	20 ms timer (nominal 50 Hz)
`terrain_classifier`	Intelligence	Compiled tree inference, verdict, abort	per feature message
`attitude_estimator`	Control	Complementary filter (Algorithm 1)	per IMU message (100 Hz)
`stabilator_controller`	Control	PID pitch stabilisation (Algorithm 2)	per attitude message (100 Hz)
`landing_gear_controller`	Control	Interlocked, rate-limited gear (Algorithm 6)	50 Hz
`flight_supervisor`	Control	Approach state machine (Section 4.8)	20 ms timer (nominal 50 Hz)
`flight_carrier` (script)	Simulation	Prescribed flight-rail trajectory	50 Hz

Table 10: ROS 2 nodes of the simulation. The last two nodes belong to the approach scenario.

6 Hardware Bench Prototype

In addition to the simulation, a physical bench prototype implements a subset of the same sensing and control ideas on real sensors [1]. Its purpose is to exercise attitude estimation, a stabiliser command, landing-readiness checks and a gear sequence on measured data, not to classify terrain. It is a deliberately reduced build: all functions run on one Arduino Uno, only one range sensor is connected, and both actuators are virtual, that is, the firmware computes and reports the commanded angles but drives no servo.

6.1 Hardware and connections

The connected hardware is listed in Table 12. The IMU is an MPU6500 breakout (GY-6500) configured at I^2C address `0x68`, and the ToF sensor is a regulated VL53L0X breakout [8] at its default address `0x29`. Both modules share the Uno's 3.3 V supply, ground and I^2C bus (Table 13); the bus runs at 400 kHz with a bounded transaction timeout (where the library supports it) so that a stuck transfer cannot block the control loop. The VDD, XSHUT and GPIO1 pins of the ToF breakout are left unconnected. Both SG90 servos remain disconnected because no suitable actuator supply is available, and the firmware's actuator output routine is deliberately empty.
The ESP32, the ultrasonic sensors, the magnetometer and additional ToF sensors are not used. The firmware contains a prepared three-ToF configuration (individual shutdown pins and re-assigned I^2C addresses) that is disabled; enabling it requires a suitable power arrangement, bus-voltage compatibility, individual shutdown wiring and a rigid sensor geometry, so it is not a working three-sensor terrain estimator. In particular, a single beam measures the range to a target and cannot give slope, surface material or a tilt-corrected altitude.

6.2 Attitude estimation and virtual stabiliser

Calibration. At start-up the IMU must be level and stationary with its $+z$ axis pointing upward. The driver estimates accelerometer and gyroscope biases, which are held in RAM and recomputed after every reset; start-up takes about 20 s plus sensor initialisation. The driver

Topic	Type	Rate	Purpose
/sensing/imu/data_raw	sensor_msgs/Imu	100 Hz	Raw accelerations and rates
/sensing/tof/*	sensor_msgs/LaserScan	nominal 50 Hz	Optical ranges (three)
/sensing/sonar/scan	sensor_msgs/LaserScan	20 Hz (model)	Acoustic ranges
/sensing/altitude_agl	Float64	nominal 50 Hz	Mean ToF range
/sensing/features	Float32MultiArray	nominal 50 Hz	Seven-element feature vector
/intelligence/verdict	Int32	per feature	0 safe, 1 slope, 2 vegetation
/intelligence/abort	Bool	per feature	Interlock override
/control/attitude	Vector3Stamped	100 Hz	Filtered roll and pitch
/control/pitch_setpoint	Float64	nominal 50 Hz	Pitch reference (supervisor)
/control/elevator_cmd	Float64	100 Hz	Stabilator command
/control/gear_deploy	Bool	nominal 50 Hz	Deploy request
/control/gear_cmd, gear_status	Float64	50 Hz	Rate-limited gear command and status
/flight/touchdown	Bool	nominal 50 Hz	Touchdown flag
/flight/supervisor_state	String	nominal 50 Hz	State label

Table 11: Selected ROS 2 topics. Nominal rates are those of the timers or of the simulated sensors; they were not measured.

reports acceleration in units of g and angular rate in degrees per second, so the accelerometer angles are formed from ratios without a conversion to m/s^2.

Complementary filter with measured time step. The firmware samples at a nominal 100 Hz and uses the elapsed time Δt_k between samples, measured with the microsecond timer, instead of a fixed period. The accelerometer angles are

$$\phi_{\mathrm{acc}} = \operatorname{atan2}(a_y, a_z), \qquad \theta_{\mathrm{acc}} = \operatorname{atan2}\bigl(-a_x, \sqrt{a_y^2 + a_z^2}\bigr), \tag{24}$$

and the filter weight is derived from a fixed time constant $\tau = 0.50\,\mathrm{s}$,

$$\alpha_k = \frac{\tau}{\tau + \Delta t_k}, \qquad \theta_k = \alpha_k\bigl(\theta_{k-1} + \omega_y\,\Delta t_k\bigr) + (1 - \alpha_k)\,\theta_{\mathrm{acc}}, \tag{25}$$

with the roll channel handled in the same way and angles wrapped to $\pm 180^\circ$. Because α_k follows Δt_k, timing jitter does not change the time constant of the filter. At 100 Hz, $\alpha \approx 0.980$ and the corner frequency is $1/(2\pi\tau) \approx 0.32\,\mathrm{Hz}$, which is lower than in the simulation ($\alpha = 0.9$, $\tau \approx 0.095\,\mathrm{s}$, about 1.7 Hz): the bench filter trusts the gyroscope for longer and therefore reacts more slowly to the accelerometer, which suppresses vibration and hand-motion noise at the cost of a slower correction of gyroscope drift. The estimate is initialised from the accelerometer at the actual tilt, and is re-initialised whenever the gap between samples exceeds 0.20 s, because the motion during the missing interval was not observed. A sample is rejected as invalid if any value

Component	Role
Arduino Uno	Acquisition, attitude estimation, readiness checks and virtual commands (sensing, intelligence and control on one board)
GY-6500 / MPU6500 IMU [9]	Three-axis acceleration and angular rate
One VL53L0X breakout	Sensor-to-target distance and measurement status
Breadboard, jumper wires	Shared power, ground and I^2C connections
USB link to a computer	Power, firmware upload and serial monitoring

Table 12: Connected hardware of the bench prototype.

Uno	MPU6500 module	VL53L0X breakout
3.3 V	Power input	VIN
GND	GND	GND
A4 (SDA)	SDA	SDA
A5 (SCL)	SCL	SCL

Table 13: Wiring of the one-ToF bench setup.

is non-finite or the acceleration magnitude is practically zero ($|\boldsymbol{a}|^2 < 10^{-6}\, g^2$); the filter state is then discarded and restarted at the next valid sample. Single precision is used throughout, which also avoids the integer overflow discussed in Section 4.1.

Virtual stabiliser command. The stabiliser command is a proportional response to the pitch error,

$$u = \mathrm{clamp}\big(90^\circ + K\, e,\ 65^\circ,\ 115^\circ\big), \qquad e = \begin{cases} 0, & |\theta_{\mathrm{ref}} - \hat{\theta}| \le 1^\circ \\ \theta_{\mathrm{ref}} - \hat{\theta}, & \text{otherwise,} \end{cases} \tag{26}$$

with $K = 1$ degree of servo angle per degree of pitch error, $\theta_{\mathrm{ref}} = 0$, a neutral angle of 90° and a travel of ±25°, the same authority as the simulated stabilator. A pitch of +10° therefore produces a command of 80°, which opposes the tilt in the sense assumed for the mechanism; the sign of K must be verified against the real linkage before a servo is connected. The ±1° deadband prevents the command from dithering around the neutral angle. If the IMU sample is invalid, the stabiliser command returns to 90° while a deployed gear command is retained. There is no integral or derivative action, no aerodynamic plant and no servo feedback: the command is displayed, not applied, and the demonstration shows the response to hand tilt, which does not prove physical stabilisation.

6.3 One-ToF landing-readiness rules

The readiness logic replaces the terrain classifier of the simulation with a short set of measured rules (Table 14). The ToF sensor runs in continuous mode with a nominal 50 ms period, and a new distance is entered into the stability window only when a distinct new result is read; polling an existing result does not count as a new sample. A reading is valid if the sensor is available, the bus transfer succeeded, the device status is valid and the result is at most 250 ms old.
The status is one of three values. UNKNOWN means that data are missing, invalid or still being collected or qualified; UNSAFE means that a measured rule has failed; READY means that all rules have been satisfied for the qualification time (Algorithm 7). A fault or a stale result empties the whole window, whereas a failed tilt, distance or spread check restarts only the qualification

Check	Demonstration setting
Sensor-to-target distance	100 to 500 mm, inclusive
Absolute pitch and roll	At most 10° each
Stability window	Ten completed, valid ToF measurements
Distance spread in the window	$\max - \min \leq 20$ mm, all samples inside the distance limits
Range freshness	At most 250 ms since the last result
Qualification time	All checks continuously satisfied for 1 s after the window is complete

Table 14: Landing-readiness rules of the bench prototype. The values are configurable demonstration settings, not aircraft operating limits.

timer. No hysteresis is implemented, so the one-second qualification is the only mechanism that delays re-entry into READY.

6.4 Latched gear sequence

Operator commands arrive over the serial link (Table 15). Deployment is accepted only while the status is READY; an early request is rejected with the current reason and is not queued. Once accepted, the virtual gear command is latched at 90° and stays latched when distance, tilt or sensor validity change: the report then shows a warning and the current readiness reason, and the gear is released only by an explicit stow command or a board reset, which starts stowed. Readiness is evaluated independently of the gear state, so the gear can be deployed while the status is UNKNOWN or UNSAFE. After readiness has been lost, a fresh qualification is required before a new deployment can be accepted, and nothing deploys automatically. All displayed angles are commands, not measured positions.

Command	Behaviour
`D`	Deploy the virtual gear to 90° while READY; otherwise reject with a reason
`S`	Stow the virtual gear at 0°, also during sensor faults
`M`	Readable serial reports at 2 Hz (default)
`V`	Raw six-value IMU frames at 20 Hz for the desktop viewer

Table 15: Serial commands of the bench prototype (lower case is also accepted).

Contrast with the simulated interlock. The two prototypes use opposite policies once the gear is down. In the simulation (Section 4.7) an unsafe verdict forces the gear to the stowed position, so that deployment is always subordinate to the classifier. In the bench prototype a deployed gear is held until the operator stows it, because the commands are virtual and the loss of a single-beam range reading, which may be caused by the target leaving the field of view, is not evidence of a hazard. The policies differ intentionally, and neither one establishes flight safety.

6.5 Firmware and desktop tools

The firmware is a single Arduino sketch that uses the FastIMU [10] and Pololu VL53L0X [11] libraries and communicates at 115 200 baud. A cooperative loop runs once per 10 ms sample interval: it reads the IMU, updates the attitude, polls the ToF sensor, evaluates the readiness

Algorithm 7 Landing-readiness evaluation (every control cycle)

Input: attitude validity, pitch θ, roll ϕ, ToF reading (distance d, validity, age), window W of the last ten distances
Output: status $\in \{\text{UNKNOWN}, \text{UNSAFE}, \text{READY}\}$

if attitude invalid **or** ToF reading invalid **or** age $> 250\,\text{ms}$ **then**
    clear W and the qualification timer; **return** UNKNOWN
end if
if $|\theta| > 10^\circ$ **or** $|\phi| > 10^\circ$ **then**
    restart timer; **return** UNSAFE
end if
if $d < 100\,\text{mm}$ **or** $d > 500\,\text{mm}$ **then**
    restart timer; **return** UNSAFE
end if
if $|W| < 10$ **then**
    restart timer; **return** UNKNOWN
end if
if $\max W - \min W > 20\,\text{mm}$ **or** any $w \in W$ outside $[100, 500]\,\text{mm}$ **then**
    restart timer; **return** UNSAFE
end if
if timer not running **then**
    start timer
end if
if timer $< 1\,\text{s}$ **then**
    return UNKNOWN ▷ qualifying
end if
return READY

rules, then processes at most sixteen command characters against these fresh readings, computes the virtual commands and, at the selected interval, emits telemetry. Unsigned subtraction of the microsecond timer remains correct across counter roll-over. Fixed text is held in flash memory so that the 2 kB of RAM of the ATmega328P are not consumed by strings. The readable report contains the gear sequence, the readiness status and its reason, pitch and roll, the range and its status, the stability-window progress, the actuator command angles and warnings.
A Python desktop application (Tkinter and `pyserial`) connects to the Uno, shows the report or a raw IMU view with a ten-sample moving average that affects only the display, and sends the deploy and stow commands; the firmware alone decides whether a deployment is allowed. Viewer frames use the format `S<ax>/<ay>/<az>/<gx>/<gy>/<gz>E`, one per line, and non-finite or malformed frames are rejected. Opening the serial port can reset the board, so the IMU must be kept level during the start-up calibration, and only one application can own the port at a time. Automated unit tests with a mocked serial connection cover frame parsing and finite-value validation, display averaging, fragmented input, serial errors and the integration of display and commands; they need no hardware.

6.6 Demonstration checks

The expected behaviour of the bench prototype for the demonstration scenarios is summarised in Table 16. The connected setup and its serial reports were exercised on hardware, but no quantitative characterisation (for example of timing, range accuracy or attitude error against a reference) was carried out, and the readiness thresholds are provisional.

Scenario	Expected behaviour
Level IMU, stable target inside the range window	READY after ten readings and one second of qualification
Deploy before READY	Rejected with a reason; the request is not saved
Deploy while READY	Gear command becomes deployed at 90°
Change distance or tilt after deployment	Readiness changes; the deployed command stays latched with a warning
Remove the target or interrupt ToF communication	UNKNOWN, stability window reset, deployed command retained
Restore acceptable readings	Fresh qualification is required; no automatic deployment
Stow command	Gear command becomes stowed at 0°
Tilt the IMU about its y axis	Stabiliser command reacts to pitch and respects its limits

Table 16: Demonstration scenarios and expected behaviour of the bench prototype.

Limits of the bench prototype. The prototype has no servo-position feedback, touchdown detection, yaw estimate, aerodynamic feedback or automatic landing control. Translational acceleration distorts the gravity reference of the accelerometer, and the gyroscope integration is intended for moderate bench tilts. The numeric validation of IMU samples cannot detect every plausible stale reading, and the ToF validation uses the raw device status rather than the full quality assessment of the manufacturer's software. A single beam cannot establish terrain slope or surface material, so a READY status expresses only that the configured bench conditions hold and must not be read as a terrain-safety classification.

7 Verification and Validation of the Simulation

7.1 Synthetic testbeds

Four Gazebo worlds provide the terrain (Table 17). The first three are static: the rig stands on a fixed mount 0.70 m above the ground, so only the sensing, classification and gear chain is exercised. Figure 6 shows the rig and Figures 7 and 8 the Beta and Gamma worlds; the Alpha world is a flat ground plane and no screenshot of it is included. The fourth world places the three terrain types in sequence for the approach scenario (Figure 9).

World	Terrain	Label
Alpha	Flat ground plane	SAFE (0)
Beta	20° inclined ramp (a 4×4 m slab) and a stepped curb block beside it	UNSAFE_SLOPE (1)
Gamma	Three box-shaped mounds and a central solid cylinder; no penetrable vegetation is modelled	UNSAFE_VEGETATION (2)
Approach	Vegetation mounds ($x = 5$ to 25 m), a 20° ramp (centred at $x = 40$ m), a curb ($x = 48$ m) and a runway ($x = 58$ to 110 m)	used by the supervisor

Table 17: Synthetic testbeds.

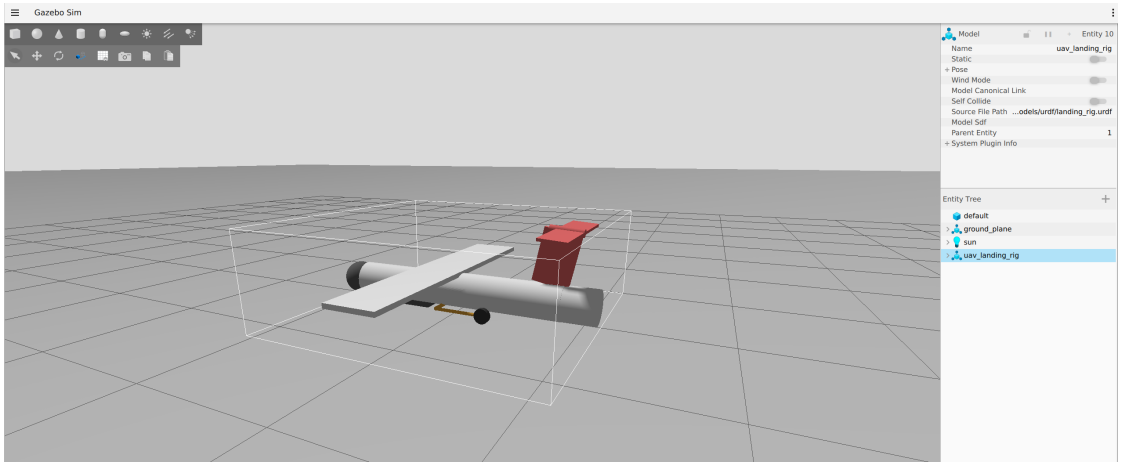


Figure 6: Gazebo view of the simulated rig: fuselage with the ventral sensor plate, high wing, T-tail with the stabilator and the retractable gear.

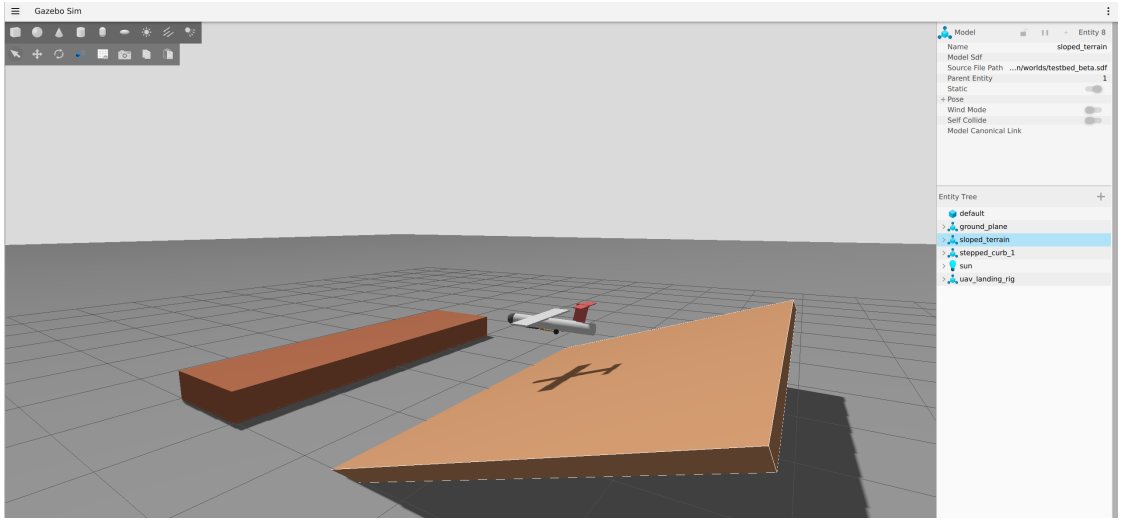


Figure 7: Beta world: the rig on its stand beside the 20° ramp slab (right) and the stepped curb block (left).

7.2 Scenario verification

The verification script launches each of Alpha, Beta and Gamma headlessly in static mode. It waits up to 60 s for ten valid scans per range sensor and fresh feature, verdict, abort and gear telemetry, and requires one second of readiness. It then sends repeated deployment requests and waits up to 15 s for the expected verdict, abort flag and gear status to hold for one second, and it writes a launch log for each world. Readiness failures are reported as failures, so fallback feature values cannot make a scenario pass. The script checks only the verdict, the abort flag and the gear status generated by the controller; it does not measure joint motion, and it does not cover the flying approach.

The live feature vectors support three observations. First, the Alpha values sit at the floors of the computation (variance $10^{-5}\,\mathrm{m}^2$, slope about $10^{-6}\,\mathrm{rad}$), far below the first threshold. Second, the Beta slope of 0.348 rad is the 20° inclination of the ramp (0.349 rad) to within $10^{-3}\,\mathrm{rad}$, which confirms the slope estimate (13) and the baseline constants on a plane, and its variance agrees with (18) as shown in Section 4.5. Third, the Gamma slope of 0.852 rad (48.8°) is a large apparent slope with a variance of $1.42 \times 10^{-3}\,\mathrm{m}^2$. Because its roll-slope component is nearly zero, this variance also agrees with $1.089 \times 10^{-3} \tan^2(0.852)$. That agreement follows algebraically from the same three range values used to calculate slope and variance; it does not establish that

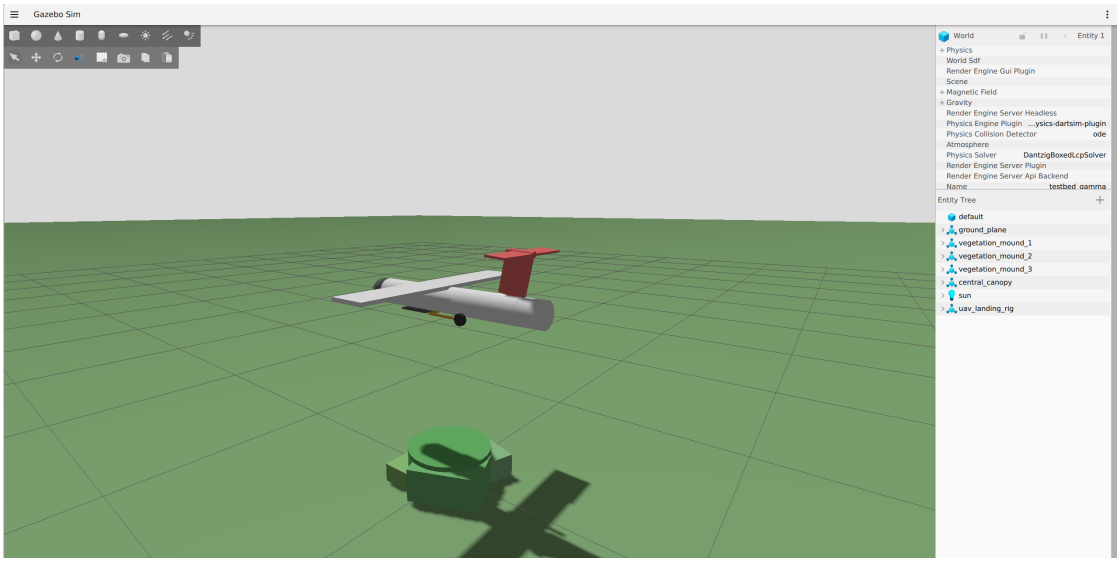


Figure 8: Gamma world: the rig above the mounds and the central cylinder.

Quantity	Alpha	Beta	Gamma
0 Mean ToF range (m)	0.730045	0.529307	0.596703
1 Spatial variance (m^2)	0.00001	0.000143	0.001422
2 Δd (m)	0.013265	0.010234	0.033453
3 Sonar scan variance (m^2)	0.000127	0.000067	0.00553
4 Total slope (rad)	0.000001	0.348175	0.851996
5 Roll slope magnitude (rad)	0.000001	0	0.000002
6 Total slope (rad)	0.000001	0.348175	0.851996
Verdict (expected, received)	0, 0	1, 1	2, 2
Abort (expected, received)	false, false	true, true	true, true
Gear status (rad)	1.5708 (deployed)	0 (stowed)	0 (stowed)
Result	passed	passed	passed

Table 18: Live feature vectors and outcomes of the verification run (Figure 10). The row numbers are the feature indices of Table 4.

the underlying mounds and canopy form a planar surface. The three runs thus fall in the three regions defined by the two thresholds. They do so with the live features, whose definitions differ from those of the training columns (Section 4.6); the verdicts are correct because the thresholds lie in the large gaps between the clusters, not because the tree was validated on the live features.

7.3 Acceptance criteria and results

The acceptance criteria and the evidence available are summarised in Table 19.

7.4 Offline checks

Offline unit tests cover the resolution of resource paths and confirm that the verification harness requests the static mode. They do not start Gazebo. The hardware tools have separate offline tests with a mocked serial port (Section 6).

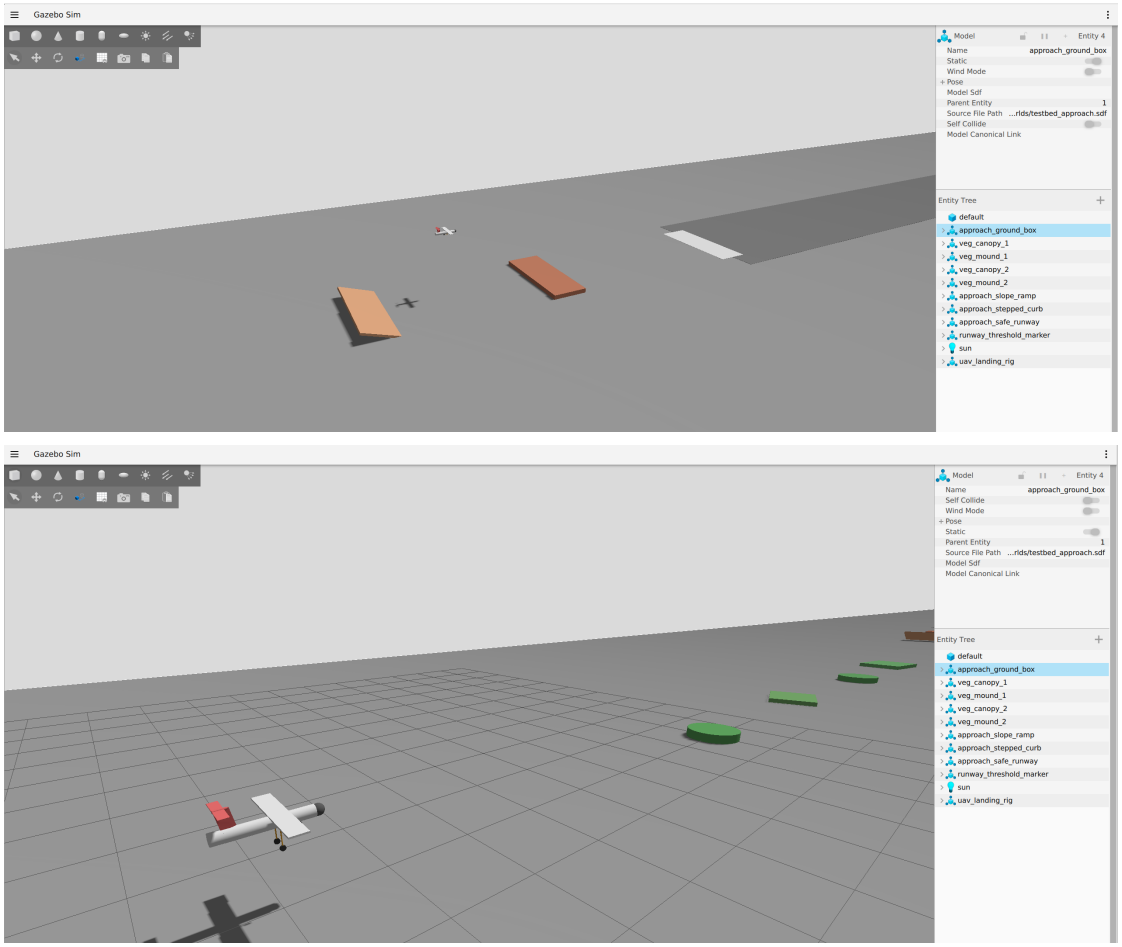


Figure 9: Approach world used with the supervisor: terrain sectors laid out along the flight corridor, seen from two viewpoints.

Metric	Target	Evidence	Status
False-safe rate	0 %	0 of 250 unsafe test samples classified safe (Table 8); Beta and Gamma produce the unsafe verdict in the live run	Met on synthetic data
Safe-terrain recall	≥ 98 %	125/125 on the test set; Alpha is classified safe in the live run	Met on synthetic data
Interlock actuation	Gear at 0 rad on abort	Generated gear status 0 on Beta and Gamma, 1.5708 rad on Alpha	Met (command level)
Inference latency	$< 1\,\mu$s	Host mean 21.98 ns, P99 102 ns, maximum 78.418 μs (Table 20)	Mean/P99 only; no worst-case bound
Pitch stability	Dynamic error $\leq 3°$	Not evaluated here; the airframe is not free to pitch	Not shown

Table 19: Acceptance criteria and the evidence in this report. The latency target is satisfied by the measured mean and P99 only; the observed maximum exceeds it, and no worst-case bound is demonstrated.

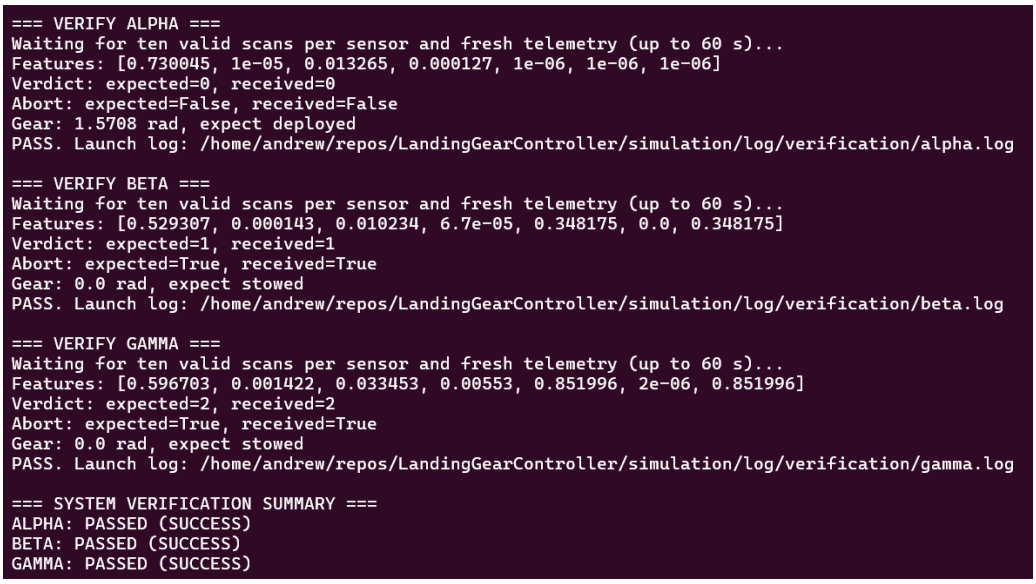


Figure 10: Terminal output of the verification script for the three static worlds.

7.5 Inference latency

The compiled classifier was benchmarked over 10^6 iterations on three synthetic feature vectors using a high-resolution clock on a host computer (Table 20, Figure 11). Two features of the result need interpretation. The minimum and median of 0 ns and the values of 101 and 102 ns at the 95th and 99th percentiles indicate that the clock advances in steps of about 100 ns, so that most evaluations, which take far less than one step, are recorded as 0 or as one step, and the mean of 21.98 ns is an average over these quantised readings. The maximum of 78 418 ns (78.418 μs) is a single outlier consistent with host scheduling and exceeds the 1 μs target. Only the measured mean and P99 satisfy that target; they do not establish a per-evaluation upper bound. The reported rate of more than 4.5×10^7 evaluations per second is an extrapolation from the mean. The benchmark does not include the ROS pipeline, servo response or a microcontroller, and it gives no worst-case execution time; it indicates only that the classifier is negligible against the 20 ms feature period.

	Min	Mean	Median	P95	P99	Max
Latency (ns)	0	21.98	0	101	102	78 418

Table 20: Inference latency over 10^6 iterations on a host computer (clock steps of about 100 ns).

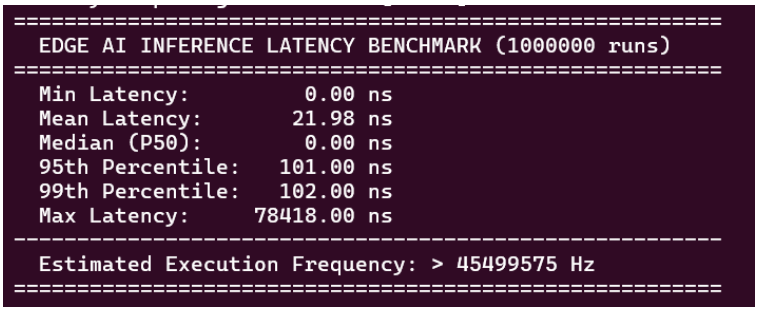


Figure 11: Terminal output of the inference benchmark.

8 Discussion

Design rationale. The central design choice is the separation of perception from control. The attitude loop runs on the IMU stream, whereas the perception workload runs on its own timer, and the two meet only through a boolean abort flag. This keeps the safety argument short: the gear can be deployed only if the interlock input is clear. The classifier is a short list of auditable comparisons rather than an opaque model.

What the results show. In the static worlds the full chain from ranges to gear status behaves as intended, the slope estimate recovers the ramp inclination, and the classifier agrees with the world labels. These results demonstrate the data path and the interlock logic on idealised terrain.

Limitations. The following limitations should be kept in mind.

- *Simulation fidelity.* The ToF sensors are noise-free, the terrains are rigid boxes and no optical penetration is modelled. Real ultrasonic transducers exhibit specular reflection from inclined surfaces, temperature-dependent speed of sound and cross-talk, and real ToF sensors are affected by ambient light and surface reflectivity. With realistic range noise, the short baselines of Section 4.3 amplify the noise to about 2° per axis for 2 mm.
- *Data and score.* The dataset consists of three tight clusters recorded with an earlier sensing pipeline, the labels are assigned per world, and the thresholds are cluster midpoints. The perfect test score is therefore optimistic, and no evidence exists on unseen or mixed terrain. The tree was not retrained for the current feature definitions, which differ from the recorded columns.
- *Class meaning.* On planar returns the second split acts as a second slope threshold, and the optical-acoustic features are not used, so the UNSAFE_VEGETATION class is not shown to detect vegetation as such.
- *Level-plate assumption.* The slope estimate uses neither the filtered attitude nor a surface normal, so body pitch and roll alias into slope, and the 0.1 m patch scale limits what it can detect.
- *No aerodynamic plant.* The airframe is held by a stand or carried on a rail, so the stabilator cannot change the body pitch. The pitch loop acts on the estimated pitch only, and the stability target of Table 19 is not demonstrated.
- *Supervisor.* The supervisor drives a prescribed trajectory, is not covered by the verification script, and has an ineffective pitch subscription (Section 4.8).
- *Timing evidence.* The latency figure is a host measurement with a coarse clock. No embedded worst-case timing, task isolation or ARINC 653 compliance is claimed.
- *Reduced hardware.* The bench prototype of Section 6 uses one Uno, an IMU and a single ToF sensor, virtual actuators and provisional thresholds; it cannot estimate terrain slope or material, and its timing, accuracy and electrical behaviour were not characterised quantitatively.

Future work. Natural extensions are (i) recording a new dataset with the current feature definitions, with noise, varied inclinations and mixed terrain, and a split by run or by geometry; (ii) a margin-based or debounced abort so that single noisy frames cannot toggle the interlock; (iii) compensation of the slope estimate with the filtered attitude, or a surface normal from the three points; (iv) a pitching-moment model so that the stabiliser loop is closed through the airframe; (v) enabling the prepared three-ToF configuration on the bench, with powered servos

once the mechanics and the sign of the stabiliser command are checked; and (vi) porting the domains to the planned boards for hardware-in-the-loop tests.

9 Conclusion

We presented a landing gear assistance system that judges whether the terrain below a descending UAV is suitable for landing and gates gear deployment accordingly. The simulation combines a three-domain architecture, a complementary-filter attitude estimator, an anti-windup PID pitch stabiliser, a slope estimate from the range differences of a triangular ToF array, a local height-variation measure and an optical-acoustic comparison, and a decision tree of two decision nodes that is compiled into allocation-free C++. A rate-limited gear controller ensures that an unsafe verdict dominates the deploy request. In three static worlds the verdict, abort flag and gear status were as expected, the measured ramp slope of 0.348 rad matched the geometry, and the classifier scored 375 of 375 on a held-out split, a result explained by tight clusters and midpoint thresholds and not a measure of performance on real terrain. The host benchmark gave a mean inference time of 21.98 ns with a clock resolution of about 100 ns. A bench prototype on an Arduino Uno confirmed that attitude estimation, a virtual stabiliser command and rule-based readiness checks operate on real IMU and ToF data with the servos disconnected. Retraining on the current features, validation on noisy and mixed terrain, and hardware-in-the-loop tests are the next steps.

References

[1] A. Manitsas and M. Vrana, *LandingGearController: UAV Landing Gear Controller, Simulation and Hardware Prototype*, software repository (GPL-3.0), https://github.com/AndrewManitsas/LandingGearController.

[2] ARINC Specification 653, *Avionics Application Software Standard Interface*, Aeronautical Radio, Inc.

[3] S. Macenski, T. Foote, B. Gerkey, C. Lalancette and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, eabm6074, 2022.

[4] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2004, pp. 2149-2154.

[5] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, Princeton University Press, 2008.

[6] L. Breiman, J. H. Friedman, R. A. Olshen and C. J. Stone, *Classification and Regression Trees*, Wadsworth, 1984.

[7] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

[8] STMicroelectronics, *VL53L0X time-of-flight ranging sensor*, datasheet, https://www.st.com/resource/en/datasheet/vl53l0x.pdf.

[9] InvenSense, *MPU-6500 Product Specification*, available from the manufacturer at https://www.invensense.tdk.com/en-us/products/inertial-sensors/mpu-6500.

[10] LiquidCGS, *FastIMU*, Arduino library for inertial measurement units, version 1.3.0 used in the bench build, https://github.com/LiquidCGS/FastIMU.

[11] Pololu Corporation, *VL53L0X library for Arduino*, version 1.3.1, https://github.com/pololu/vl53l0x-arduino.

[12] *HC-SR04 ultrasonic ranging module*, manufacturer datasheet.

[13] InvenSense, *MPU-9250 Product Specification*.

[14] Raspberry Pi Ltd, *Raspberry Pi Pico 2 W and RP2350 datasheet*.

[15] Microchip Technology, *ATmega328P datasheet*.

[16] TowerPro, *SG90 micro servo*, manufacturer datasheet.